



Universidad de las Américas Puebla (UDLAP)

Licenciatura en Ciencia de Datos

Actividad 6: Sincronización de procesos e hilos en Java

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: 17 de septiembre de 2026

Índice general

Índice de figuras	III
1. Sincronización de hilos con <code>join()</code>	1
2. Sincronización de procesos con <code>waitFor()</code>	4
3. Análisis y conclusiones	8
Referencias	9

Índice de figuras

1.1. Código Java y salida del programa de sincronización de hilos con <code>join()</code>	3
2.1. Proceso hijo con <code>ProcessBuilder</code>	6
2.2. Proceso principal con <code>waitFor()</code>	7

1. Sincronización de hilos con `join()`

Desarrollen un programa en Java que implemente tres tareas utilizando Runnable: el Hilo 1 debe simular la obtención de datos, el Hilo 2 el procesamiento de los datos y el Hilo 3 la generación de un reporte. Utilicen `start()` y `join()` para garantizar el orden Hilo 1, Hilo 2 y Hilo 3. Cada hilo debe mostrar en consola cuándo inicia y cuándo termina su tarea.

Entradas. Tres objetos que implementan la interfaz Runnable, uno para obtener datos, otro para procesarlos y otro para generar el reporte. También se utilizan tres objetos Thread y dos datos compartidos entre las tareas consecutivas.

Procesamiento. El proceso principal crea los tres hilos. Inicia el Hilo 1 con `start()` y ejecuta `join()` para esperar su finalización. Después inicia el Hilo 2 y vuelve a esperar con `join()`. Finalmente inicia el Hilo 3 y espera su terminación. De esta manera, la finalización de una tarea se convierte en la condición para iniciar la siguiente.

Salidas. Cada hilo muestra un mensaje de inicio y otro de terminación. El Hilo 1 deja disponibles los datos, el Hilo 2 muestra que fueron procesados y el Hilo 3 genera el reporte. El mensaje final confirma que las tres tareas terminaron en el orden solicitado.

Estructuras de control utilizadas. Se emplean tres clases que implementan Runnable, tres objetos Thread, llamadas a `start()` para iniciar hilos y llamadas a `join()` para esperar su terminación. También se utilizan métodos `run()`, asignaciones de datos compartidos y un bloque try-catch para atender una posible interrupción durante la pausa de cada tarea.

Conceptos. En Java, Runnable representa la tarea que puede ejecutarse en un hilo. `start()` inicia un nuevo hilo de ejecución y provoca que la JVM invoque su método `run()`. `join()` hace que el hilo que ejecuta esa

Ejercicio 1. Sincronización de hilos con `join()`

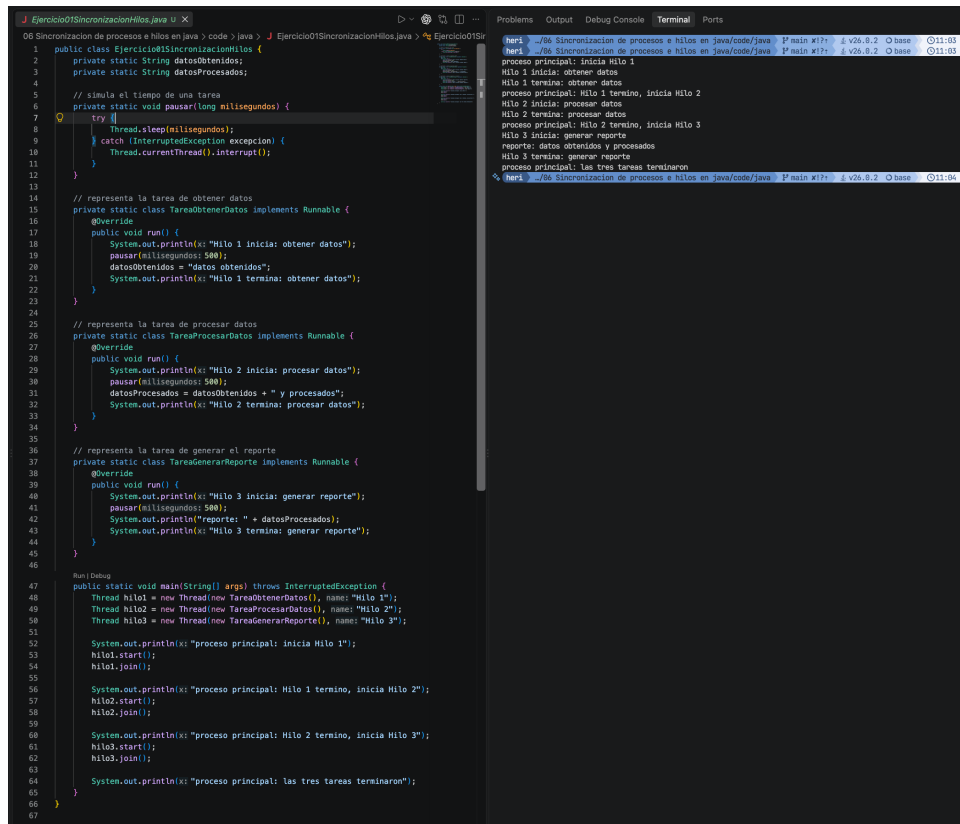
instrucción espere hasta que termine el hilo indicado. En este ejemplo, la espera establece una dependencia: procesar depende de que la obtención de datos haya terminado y generar el reporte depende de que el procesamiento haya terminado.

Prueba de escritorio. La siguiente secuencia muestra el orden de ejecución establecido por las llamadas a `start()` y `join()`.

Paso	Hilo o proceso	Acción y estado
1	Proceso principal	Crea las tres tareas <code>Runnable</code> y los tres objetos <code>Thread</code> .
2	Hilo 1	<code>start()</code> inicia la obtención de datos; el hilo muestra que inicia.
3	Proceso principal	Ejecuta <code>hilo1.join()</code> y permanece en espera hasta que el Hilo 1 termina.
4	Hilo 1	Completa la obtención, guarda los datos y muestra que termina.
5	Hilo 2	El proceso principal continúa, ejecuta <code>start()</code> y el Hilo 2 inicia el procesamiento.
6	Proceso principal	Ejecuta <code>hilo2.join()</code> y espera la terminación del Hilo 2.
7	Hilo 2	Procesa los datos y muestra que termina.
8	Hilo 3	El proceso principal ejecuta <code>start()</code> ; el Hilo 3 genera el reporte.
9	Proceso principal	<code>hilo3.join()</code> espera la terminación del Hilo 3 y después continúa.

¿Qué ocurriría si se eliminaran los `join()`? Los tres hilos podrían iniciar sin un orden garantizado. El Hilo 2 podría intentar usar los datos antes de que el Hilo 1 los obtenga, y el Hilo 3 podría generar el reporte antes de que el procesamiento termine. Además, el proceso principal podría mostrar el mensaje final antes de que las tareas concluyeran. Se perdería la dependencia entre las tareas.

Ejercicio 1. Sincronización de hilos con join()



```
06 Sincronización de procesos e hilos en java > code > java > Ejercicio01SincronizaciónHilos > Ejercicio01SincronizaciónHilos.java
1 public class Ejercicio01SincronizaciónHilos {
2     private static String datosObtenidos;
3     private static String datosProcesados;
4
5     // simula el tiempo de una tarea
6     private static void pausar(long milisegundos) {
7         try {
8             Thread.sleep(milisegundos);
9         } catch (InterruptedException excepcion) {
10             Thread.currentThread().interrupt();
11         }
12     }
13
14     // representa la tarea de obtener datos
15     private static class TareaObtenerDatos implements Runnable {
16         @Override
17         public void run() {
18             System.out.println("Hilo 1 inicia: obtener datos");
19             pausar(milisegundos: 500);
20             datosObtenidos = "datos obtenidos";
21             System.out.println("Hilo 1 termina: obtener datos");
22         }
23     }
24
25     // representa la tarea de procesar datos
26     private static class TareaProcesarDatos implements Runnable {
27         @Override
28         public void run() {
29             System.out.println("Hilo 2 inicia: procesar datos");
30             pausar(milisegundos: 500);
31             datosProcesados = datosObtenidos + " y procesados";
32             System.out.println("Hilo 2 termina: procesar datos");
33         }
34     }
35
36     // representa la tarea de generar el reporte
37     private static class TareaGenerarReporte implements Runnable {
38         @Override
39         public void run() {
40             System.out.println("Hilo 3 inicia: generar reporte");
41             pausar(milisegundos: 500);
42             System.out.println("Reporte: " + datosProcesados);
43             System.out.println("Hilo 3 termina: generar reporte");
44         }
45     }
46
47     Run | Debug
48     public static void main(String[] args) throws InterruptedException {
49         Thread hilo1 = new Thread(new TareaObtenerDatos(), name: "Hilo 1");
50         Thread hilo2 = new Thread(new TareaProcesarDatos(), name: "Hilo 2");
51         Thread hilo3 = new Thread(new TareaGenerarReporte(), name: "Hilo 3");
52
53         System.out.println("Proceso principal: inicia Hilo 1");
54         hilo1.start();
55         hilo1.join();
56
57         System.out.println("Proceso principal: Hilo 1 termino, inicia Hilo 2");
58         hilo2.start();
59         hilo2.join();
60
61         System.out.println("Proceso principal: Hilo 2 termino, inicia Hilo 3");
62         hilo3.start();
63         hilo3.join();
64
65         System.out.println("Proceso principal: las tres tareas terminaron");
66     }
67 }
```

```
h1: _/06 Sincronización de procesos e hilos en java/code/java P main x17: J v26.0.2 O base C11:05
h2: _/06 Sincronización de procesos e hilos en java/code/java P main x17: J v26.0.2 O base C11:05
Proceso principal: inicia Hilo 1
Hilo 1 inicia: obtener datos
Hilo 1 termina: obtener datos
Proceso principal: Hilo 1 termino, inicia Hilo 2
Hilo 2 inicia: procesar datos
Hilo 2 termina: procesar datos
Proceso principal: Hilo 2 termino, inicia Hilo 3
Hilo 3 inicia: generar reporte
Reporte: datos obtenidos y procesados
Hilo 3 termina: generar reporte
Proceso principal: las tres tareas terminaron
h3: _/06 Sincronización de procesos e hilos en java/code/java P main x17: J v26.0.2 O base C11:06
```

Figura 1.1: Código Java y salida del programa de sincronización de hilos con join().

2. Sincronización de procesos con `waitFor()`

Desarrollen un segundo ejemplo en Java utilizando `ProcessBuilder` para iniciar un proceso externo. El proceso principal debe crear e iniciar el proceso, mostrar que fue iniciado, utilizar `waitFor()` para esperar su finalización, mostrar que terminó y continuar con la ejecución del programa.

Entradas. El proceso principal recibe la ruta del ejecutable de Java de la JVM actual y la ruta del archivo Java que contiene el proceso hijo. También utiliza un objeto `ProcessBuilder` para describir el proceso externo.

Procesamiento. El proceso principal crea el `ProcessBuilder`, conecta la entrada y salida del proceso hijo con el terminal e inicia el proceso mediante `start()`. Después ejecuta `waitFor()` y queda en espera hasta que el proceso hijo termina. Cuando la espera concluye, el programa principal obtiene el código de salida y continúa con un mensaje final.

Salidas. El terminal muestra que el proceso principal crea e inicia el proceso hijo, que el proceso hijo inicia y termina su tarea, y que el proceso principal continúa después de `waitFor()`. El código de salida igual a cero indica que el proceso hijo terminó correctamente.

Estructuras de control utilizadas. Se emplean las clases `ProcessBuilder` y `Process`, los métodos `start()`, `inheritIO()` y `waitFor()`, además de un bloque `try-catch` mediante las excepciones declaradas en `main`. El proceso hijo utiliza una pausa para hacer visible el periodo en que el proceso principal permanece esperando.

Conceptos. `ProcessBuilder` permite crear un proceso externo y `start()` lo inicia. `waitFor()` hace que el proceso principal espere hasta que el proceso hijo termine y devuelve su código de salida. Esta es una coordinación entre procesos: el proceso principal y el proceso hijo son ejecuciones separadas, por lo que no comparten directamente las variables del programa principal.

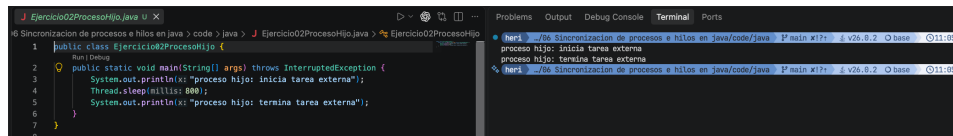
Ejercicio 2. Sincronización de procesos con `waitFor()`

Prueba de escritorio. La siguiente secuencia representa el flujo solicitado, incluida la espera del proceso principal.

Paso	Proceso	Acción y estado
1	Proceso principal	Crea el objeto <code>ProcessBuilder</code> con el ejecutable y el archivo Java del proceso hijo.
2	Proceso principal	Ejecuta <code>start()</code> y muestra que el proceso hijo fue iniciado.
3	Proceso hijo	Inicia su tarea externa y muestra el mensaje correspondiente.
4	Proceso principal	Ejecuta <code>waitFor()</code> y permanece en espera bloqueada hasta la finalización del proceso hijo.
5	Proceso hijo	Termina su tarea y devuelve el código de salida cero.
6	Proceso principal	<code>waitFor()</code> termina; el proceso principal muestra el código de salida.
7	Proceso principal	Continúa con la ejecución y muestra el mensaje final.

¿Qué función cumple `waitFor()`? Coordina al proceso principal con el proceso hijo. El proceso principal no continúa después de esa llamada hasta que el proceso hijo termina; además, la llamada devuelve el código de salida del proceso hijo.

Ejercicio 2. Sincronización de procesos con waitFor()



The screenshot shows an IDE with a Java file named `Ejercicio02ProcesoHijo.java`. The code defines a `main` method that prints "proceso hijo: inicia tarea externa", sleeps for 800ms, and then prints "proceso hijo: termina tarea externa". The `Terminal` tab on the right shows the execution output, which matches the code's print statements, indicating successful execution.

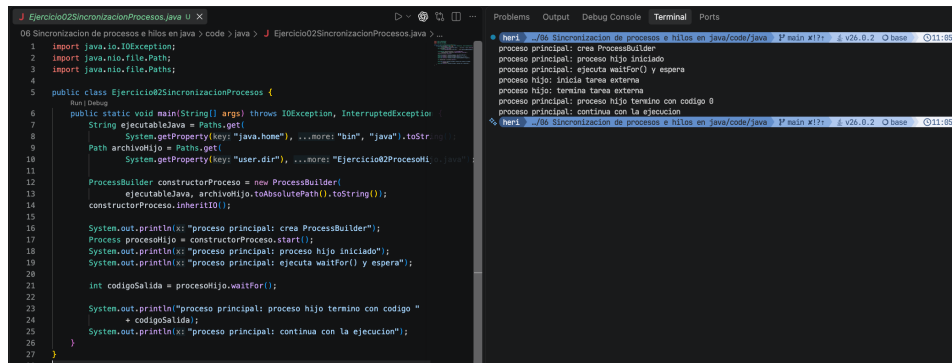
```
1 public class Ejercicio02ProcesoHijo {  
2     public static void main(String[] args) throws InterruptedException {  
3         System.out.println("proceso hijo: inicia tarea externa");  
4         Thread.sleep(800);  
5         System.out.println("proceso hijo: termina tarea externa");  
6     }  
7 }
```

Terminal Output:

```
proceso hijo: inicia tarea externa  
proceso hijo: termina tarea externa
```

Figura 2.1: Proceso hijo con ProcessBuilder.

Ejercicio 2. Sincronización de procesos con waitFor()



The image shows a screenshot of an IDE with two panels. The left panel displays the source code for a Java class named `Ejercicio02SincronizacionProcesos`. The code implements a main method that creates a `ProcessBuilder` to start a child process, then calls `waitFor()` to wait for its completion before continuing. The right panel shows the output of the program, which prints the sequence of events: creating the process builder, starting the child process, waiting for it to finish, and then continuing the main process.

```
1 import java.io.IOException;
2 import java.nio.file.Path;
3 import java.nio.file.Paths;
4
5 public class Ejercicio02SincronizacionProcesos {
6     public static void main(String[] args) throws IOException, InterruptedException {
7         String ejecutableJava = Paths.get(
8             System.getProperty(key: "java.home"), ...more: "bin", "java").toString();
9         Path archivoHijo = Paths.get(
10             System.getProperty(key: "user.dir"), ...more: "Ejercicio02ProcesoHijo.java");
11
12         ProcessBuilder constructorProceso = new ProcessBuilder(
13             ejecutableJava, archivoHijo.toAbsolutePath().toString());
14         constructorProceso.inheritIO();
15
16         System.out.println("proceso principal: crea ProcessBuilder");
17         Process procesoHijo = constructorProceso.start();
18         System.out.println("proceso principal: proceso hijo iniciado");
19         System.out.println("proceso principal: ejecuta waitfor() y espera");
20
21         int codigoSalida = procesoHijo.waitFor();
22
23         System.out.println("proceso principal: proceso hijo termino con codigo "
24             + codigoSalida);
25         System.out.println("proceso principal: continua con la ejecucion");
26     }
27 }
```

Output:

```
proceso principal: crea ProcessBuilder
proceso principal: proceso hijo iniciado
proceso principal: ejecuta waitfor() y espera
proceso hijo: inicia tarea externa
proceso hijo: termina tarea externa
proceso principal: proceso hijo termino con codigo 0
proceso principal: continua con la ejecucion
```

Figura 2.2: Proceso principal con waitFor().

3. Análisis y conclusiones

¿Qué función cumple join()? join() establece una dependencia entre hilos. El hilo que ejecuta la llamada espera la terminación del hilo indicado y después puede continuar. En el primer programa permite que la obtención de datos termine antes de iniciar el procesamiento, y que el procesamiento termine antes de generar el reporte.

¿Qué diferencia existe entre sincronizar hilos y procesos en los ejemplos realizados? Los hilos pertenecen al mismo programa y se ejecutan dentro de la misma JVM. En el primer ejemplo, join() ordena tareas realizadas por objetos Runnable y permite que las tareas consecutivas utilicen los datos compartidos. En el segundo ejemplo, ProcessBuilder inicia otra ejecución de Java; waitFor() coordina su finalización y el proceso principal continúa cuando recibe el código de salida.

Espera. En ambos casos el hilo o proceso principal debe esperar un evento: la terminación de otro hilo o la terminación de otro proceso. Esta espera establece el orden de ejecución requerido y evita que una tarea dependiente continúe antes de tiempo.

Conclusión. Los dos programas muestran que la sincronización coordina la ejecución de procesos e hilos para que las operaciones ocurran en el orden adecuado. start() permite iniciar una nueva ejecución, mientras que join() y waitFor() establecen el punto en el que la ejecución debe esperar. En el primer programa, eliminar join() haría que el orden de las tareas dependiera del planificador y podría producir un reporte sin datos procesados. En el segundo programa, eliminar waitFor() permitiría que el proceso principal continuara antes de que terminara el proceso hijo.

Bibliografía

- [1] L. Lamport, *LaTeX: A Document Preparation System*, 2nd ed. Reading, Massachusetts: Addison-Wesley, 1994.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 7th ed. Boston, MA: Prentice Hall, 2011.